

BIOS 640 Week 2: Preparing NHANES Blood Pressure and Demographic Data

Urvashi

October 03, 2026

1 Introduction

The National Health and Nutrition Examination Survey (NHANES) is a program of studies that assesses the health and nutritional status of adults and children in the United States, combining interviews with physical examinations and laboratory data. This report documents how I prepared **blood pressure** and **demographic** data from three survey cycles (waves): 2013-2014, 2015-2016 and 2017-2018.

The workflow follows the sections below: *Data import and exploration*, *Merging*, *Cleaning and recoding*, and *Filtering and reshaping*. Names use `snake_case` and paths use `here::here()`.

2 Data import and exploration

Each wave contributes two files, a demographic file and a blood pressure file, so six files are imported in total. Names follow a `type_year` pattern so that any dataset can be identified from its name alone.

```
# Import datasets directly from local project directories using the specified naming convention
demo_2013 <- import(here("data", "raw", "2013-2014_DEMO_H.xpt.txt"), format = "xpt")
demo_2015 <- import(here("data", "raw", "2015-2016_DEMO_I.xpt.txt"), format = "xpt")
demo_2017 <- import(here("data", "raw", "2017-2018_DEMO_J.xpt.txt"), format = "xpt")

bp_2013 <- import(here("data", "raw", "2013-2014_BPX_H.xpt.txt"), format = "xpt")
bp_2015 <- import(here("data", "raw", "2015-2016_BPX_I.xpt.txt"), format = "xpt")
bp_2017 <- import(here("data", "raw", "2017-2018_BPX_J.xpt.txt"), format = "xpt")
```

2.1 Dimensions and variable types

```
raw_datasets <- list(
  demo_2013 = demo_2013, demo_2015 = demo_2015, demo_2017 = demo_2017,
  bp_2013   = bp_2013,   bp_2015   = bp_2015,   bp_2017   = bp_2017
)

tibble(
  dataset = names(raw_datasets),
  rows    = map_int(raw_datasets, nrow),
  columns = map_int(raw_datasets, ncol),
  types   = map_chr(raw_datasets,
    \ (d) paste(sort(unique(map_chr(d, \ (x) class(x)[1])))),
```

```
collapse = ", ")
) %>% kable(caption = "Dimensions and variable types of the six raw datasets")
```

Table 1: Dimensions and variable types of the six raw datasets

dataset	rows	columns	types
demo_2013	10175	47	numeric
demo_2015	9971	47	numeric
demo_2017	9254	46	numeric
bp_2013	9813	23	numeric
bp_2015	9544	21	numeric
bp_2017	8704	21	numeric

Every variable in every file arrives as **numeric**. This is a property of the .XPT transport format, which stores only numbers and character strings and has no concept of a factor, so all of NHANES' categorical codes import as plain numbers.

2.2 Variables stored in an inappropriate format

Two examples where the imported type does not match what the variable actually represents:

- **SEQN (participant ID)**: stored as numeric, but it is an identifier rather than a quantity. Its mean is meaningless and nothing is gained by ordering participants by it. A **character** type is more suitable, because it makes accidental arithmetic impossible.
- **RIAGENDR (gender)**: stored as numeric codes (1 = male, 2 = female) for a nominal category. Treating it as a number invites a nonsensical “mean gender” of 1.5. A labelled **factor** is more suitable, because it carries the meaning of each code and restricts the variable to its valid values. The same argument applies to the ethnicity, education, marital status, family income and family size variables.

3 Merging the datasets

I kept only the required columns, stacked the three waves with a `wave_id` column, and joined the two combined datasets. A **full join** keeps every participant who appears in at least one of the two combined datasets.

```
bp_vars <- c("SEQN", "PEASCCT1",
             paste0(rep(c("BPXSY", "BPXDI"), 4), rep(1:4, each = 2)))
demo_vars <- c("SEQN", "RIAGENDR", "RIDAGEYR", "RIDRETH1", "RIDRETH3",
               "DMDEDUC2", "DMDMARTL", "DMDFMSIZ", "INDFMIN2")

# One helper used for both file types: select the required columns from each
# wave, then stack them, letting bind_rows() write the wave label itself.
stack_waves <- function(datasets, vars) {
  datasets %>% map(\(d) select(d, all_of(vars))) %>% bind_rows(.id = "wave_id")
}

bp_all <- stack_waves(list("2013-2014" = bp_2013, "2015-2016" = bp_2015,
                           "2017-2018" = bp_2017), bp_vars)
demo_all <- stack_waves(list("2013-2014" = demo_2013, "2015-2016" = demo_2015,
                             "2017-2018" = demo_2017), demo_vars)
```

```
# Joining on wave_id as well as SEQN keeps a single wave_id column in the
# result, instead of wave_id.x and wave_id.y.
nhanes_merged <- full_join(bp_all, demo_all, by = c("SEQN", "wave_id"))

n_both      <- nrow(inner_join(bp_all, demo_all, by = c("SEQN", "wave_id")))
n_only_bp   <- nrow(anti_join(bp_all, demo_all, by = c("SEQN", "wave_id")))
n_only_demo <- nrow(anti_join(demo_all, bp_all, by = c("SEQN", "wave_id")))

tibble(dataset = c("bp_all", "demo_all", "nhanes_merged"),
       rows    = c(nrow(bp_all), nrow(demo_all), nrow(nhanes_merged)),
       columns  = c(ncol(bp_all), ncol(demo_all), ncol(nhanes_merged))) %>%
  kable(caption = "Combined datasets versus the merged dataset")
```

Table 2: Combined datasets versus the merged dataset

dataset	rows	columns
bp_all	28061	11
demo_all	29400	10
nhanes_merged	29400	19

The merged dataset has **29400 rows and 19 columns**.

Rows. The merged row count is not the sum of the two inputs. It equals the row count of `demo_all` exactly, because 0 participants appear in the blood pressure data without appearing in the demographic data. That zero is the informative result: every person with a blood pressure reading also has a demographic record, with no exceptions. This follows from how NHANES is run. All participants are interviewed, which produces the demographic file, and only a subset then attends the mobile examination centre where blood pressure is measured. The blood pressure file is therefore a strict subset of the demographic file. The remaining 1339 participants were interviewed but never examined, so they enter the merged data with `NA` in every blood pressure column.

Columns. The merged column count is $11 + 10 - 2 = 19$. Two columns are subtracted rather than one because both `SEQN` and `wave_id` were used as join keys, so each appears once in the result instead of twice.

4 Cleaning and recoding

4.1 Step 1: Review the merged data

```
skim_without_charts(nhanes_merged) %>%
  yank("numeric") %>%
  select(skim_variable, n_missing, complete_rate, mean, p0, p100) %>%
  kable(digits = 3)
```

skim_variable	n_missing	complete_rate	mean	p0	p100
SEQN	0	1.000	88256.500	73557	102956
PEASCCT1	28400	0.034	20.638	1	122
BPXSY1	8781	0.701	119.941	66	236
BPXDI1	8781	0.701	66.547	0	136
BPXSY2	8115	0.724	119.991	66	238

skim_variable	n_missing	complete_rate	mean	p0	p100
BPXDI2	8115	0.724	66.463	0	144
BPXSY3	8178	0.722	119.665	62	238
BPXDI3	8178	0.722	66.363	0	140
BPXSY4	28005	0.047	128.179	74	234
BPXDI4	28005	0.047	71.299	0	132
RIAGENDR	0	1.000	1.508	1	2
RIDAGEYR	0	1.000	32.522	0	80
RIDRETH1	0	1.000	3.108	1	5
RIDRETH3	0	1.000	3.329	1	7
DMDEDUC2	12343	0.580	3.495	1	9
DMDMARTL	12343	0.580	2.637	1	99
DMDFMSIZ	0	1.000	3.701	1	7
INDFMIN2	926	0.969	11.282	1	99

```
nhanes_merged %>%
  summarise(across(matches("^BPXDI[1-4]$", \x) sum(x == 0, na.rm = TRUE))) %>%
  pivot_longer(everything(), names_to = "reading", values_to = "readings_of_zero") %>%
  kable(caption = "Diastolic readings recorded as zero")
```

Table 3: Diastolic readings recorded as zero

reading	readings_of_zero
BPXDI1	275
BPXDI2	325
BPXDI3	367
BPXDI4	23

Four observations from this summary.

Every variable except `wave_id` is numeric, including the participant ID and every categorical code. `wave_id` is character because it was created from the names of the list passed to `bind_rows()`.

Missingness in the blood pressure readings is not uniform. The first three systolic and diastolic readings have broadly similar completeness, and the fourth is missing for the overwhelming majority of participants. This is by design rather than a data quality problem: a fourth reading is taken only when one of the earlier readings was interrupted or incomplete.

DMDEDUC2 and DMDMARTL are missing for participants under 20. The codebook labels DMDEDUC2 as “Education level - Adults 20+”, and both questions are asked only of adults, so the missingness is structural and should not be treated as data loss.

Several variables carry reserved codes that are not real values. DMDEDUC2 uses 7 for “refused” and 9 for “don’t know”, while DMDMARTL and INDFMIN2 use 77 and 99 for the same two answers. Left in place, a “don’t know” income would enter any calculation as the number 99. These have to become NA. RIAGENDR, RIDRETH1, RIDRETH3 and DMDFMSIZ have no such codes.

The minimum diastolic reading is zero. All four diastolic columns have a `p0` of 0, which is not a physiologically meaningful blood pressure. The NHANES documentation states, under Data Processing and Editing, that diastolic blood pressure can be zero, but gives no reason for it. The zeros are therefore deliberate and documented rather than an import error.

One further quirk worth noting for later: RIDAGEYR is **topcoded**, so every participant aged 80 or older is recorded as exactly 80.

4.2 Step 2: Clean the column names

`clean_names()` standardises everything to `snake_case`. That alone still leaves names like `riagendr` and `bpxsy1`, which are meaningless to a reader, so the codebook names are then renamed to describe what they hold.

```
nhanes_clean <- nhanes_merged %>%
  clean_names() %>%
  rename(participant_id = seqn, gender = riagendr, age_years = ridageyr,
         race_ethnicity_5cat = ridreth1, race_ethnicity_6cat = ridreth3,
         education = dmddeduc2, marital_status = dmdmart1,
         family_size = dmdfmsiz, family_income = indfmin2,
         bp_comment = peascct1) %>%
  rename_with(\(x) str_replace(x, "~bpxsy(\\d)$", "sbp_\\1"), starts_with("bpxsy")) %>%
  rename_with(\(x) str_replace(x, "~bpxdi(\\d)$", "dbp_\\1"), starts_with("bpxdi"))
```

4.3 Step 3: Recode the variables

Each categorical variable becomes a labelled factor using its NHANES codebook. Listing only the valid codes in `levels` means the reserved codes identified in Step 1 are converted to NA automatically, with no separate filtering step.

Seven variables are recoded (gender, two ethnicity variables, education, marital status, family income and family size) and an age grouping is derived. Two representative examples are shown; the full chunk is in the `.Rmd`.

```
gender      = factor(gender, levels = c(1, 2), labels = c("Male", "Female"))
age_group   = cut(age_years, breaks = c(0, 18, 30, 45, 60, 75, Inf),
                 labels = c("0-17", "18-29", "30-44", "45-59", "60-74", "75+"),
                 right = FALSE)
```

The age bands follow recognisable life stages rather than equal-width intervals: minors, young adults, early middle age, late middle age, early old age, and the topcoded group.

```
nhanes_clean %>%
  count(age_group, gender) %>%
  pivot_wider(names_from = gender, values_from = n) %>%
  kable(caption = "Participants by age group and gender after recoding")
```

Table 4: Participants by age group and gender after recoding

age_group	Male	Female
0-17	5809	5630
18-29	1771	1891
30-44	1983	2284
45-59	1981	2159
60-74	2006	2027
75+	902	957

```
nhanes_clean %>%
  count(education) %>%
  kable(caption = "Education level after recoding, including reserved codes now missing")
```

Table 5: Education level after recoding, including reserved codes now missing

education	n
Less than 9th grade	1622
9th-11th grade	2105
High school graduate/GED	3864
Some college/AA degree	5240
College graduate or above	4201
NA	12368

Table 4 shows both variables carrying labels rather than the codes 1 and table 5 shows the reserved codes 7 and 9 have become NA rather than being counted as if they were education levels.

4.4 Step 4: Duplicates, invalid readings and export

```
n_duplicates <- sum(duplicated(nhanes_clean))
nhanes_clean <- distinct(nhanes_clean)
```

0 duplicate rows were found. This is the expected result: the join key was the pair `SEQN` and `wave_id`, and NHANES issues each participant a unique `SEQN` within a wave, so every row is already a distinct participant-wave combination.

`bp_comment` (PEASCCT1 in the codebook) is populated **only** when something prevented the blood pressure examination from being completed normally, and is missing for everyone whose examination went ahead as intended. The specific codes differ between waves: 2013-2014 uses a detailed twelve-code scheme, while the two later waves use four broader categories, and the same number does not always mean the same thing across them.

Testing for a missing value rather than for particular code numbers therefore identifies flagged examinations correctly in all three waves without depending on any wave's coding scheme.

```
nhanes_clean <- nhanes_clean %>%
  mutate(bp_invalid = !is.na(bp_comment)) %>%
  mutate(across(matches("(sbp|dbp)_[1-4]$"),
    \(x) if_else(bp_invalid, NA_real_, x)))
n_flagged <- sum(nhanes_clean$bp_invalid)
```

1000 examinations were flagged and had their readings set to missing.

```
dir.create(here("data", "clean"), recursive = TRUE, showWarnings = FALSE)
export(nhanes_clean, here("data", "clean", "nhanes_bp_demographics_clean.rds"))
```

The cleaned dataset is exported as `.rds` rather than `.csv` because `.rds` preserves factor levels, orderings and types, so a later analysis can import it and start work immediately rather than repeating this whole pipeline.

As a final check, the first rows of the cleaned dataset:

```
nhanes_clean %>%
  select(participant_id, wave_id, gender, age_group, education, sbp_1, dbp_1) %>%
  head(5) %>%
  kable(caption = "First five rows of the cleaned dataset")
```

Table 6: First five rows of the cleaned dataset

participant_id	wave_id	gender	age_group	education	sbp_1	dbp_1
73557	2013-2014	Male	60-74	High school graduate/GED	122	72
73558	2013-2014	Male	45-59	High school graduate/GED	156	62
73559	2013-2014	Male	60-74	Some college/AA degree	140	90
73560	2013-2014	Male	0-17	NA	108	38
73561	2013-2014	Female	60-74	College graduate or above	136	86

5 Filtering and reshaping

I kept adults with a non-missing age of 21 or older, along with age, participant ID, gender and the eight blood pressure measurements.

```
nhanes_adults <- nhanes_clean %>%
  filter(!is.na(age_years), age_years >= 21) %>%
  select(age_years, participant_id, gender, matches("(sbp|dbp)_[1-4]$"))
```

The result is in **wide format**. Each participant occupies exactly one row, and their repeated measurements are spread sideways across eight separate columns (`sbp_1` to `sbp_4` and `dbp_1` to `dbp_4`). The reading number is encoded in the column names rather than stored as data, which is the defining characteristic of wide format.

`pivot_longer()` converts it to **long format**, giving one row per participant per reading. Each of the eight column names encodes two separate facts, the measure and the reading number, so `names_to = c(".value", "reading_number")` splits them: `.value` sends `sbp` and `dbp` to their own columns, while the trailing digit becomes a `reading_number` variable.

```
nhanes_adults_long <- nhanes_adults %>%
  pivot_longer(cols = matches("(sbp|dbp)_[1-4]$"),
    names_to = c(".value", "reading_number"), names_sep = "_")

c(wide_rows = nrow(nhanes_adults), long_rows = nrow(nhanes_adults_long))
```

```
#> wide_rows long_rows
#>      16787      67148
```

```
head(nhanes_adults_long, 4)
```

```
#> # A tibble: 4 x 6
#>   age_years participant_id gender reading_number sbp dbp
#>   <dbl> <chr>          <fct> <chr>          <dbl> <dbl>
#> 1     69 73557      Male     1             122    72
#> 2     69 73557      Male     2             114    76
#> 3     69 73557      Male     3             102    74
#> 4     69 73557      Male     4              NA     NA
```

The long dataset has four times as many rows as the wide one, one per reading, and the eight measurement columns collapse into `reading_number`, `sbp` and `dbp`.